

개발된 LLM 연동 웹 어플리케이션

1. 프로젝트 개요

- 프로젝트명 : A.J.C
- 작성일 : 2026-02-05

2. 시스템 개요

- 목표 : 본 프로젝트는 업무 과정에서 발생하는 파편화된 문서들을 유기적으로 연결하는 플랫폼 구축을 목표로 합니다. 문서를 파일 단위가 아닌 의미 단위의 섹션 단위로 관리하여, 단 한 번의 수정으로 모든 관련 내용을 포함한 문서가 실시간 동기화되는 차세대 지식 관리 환경을 제공합니다.
- 기능
 - 업로드된 문서의 자동 분할 및 요약-색인 생성
 - 문서가 수정되었을 때 관련 문서의 관련 내용 자동 수정
 - 민감 정보 자동 암호화 및 저장

3. 시스템 아키텍처

- 백엔드
 - 언어 : Python
 - 프레임워크 : FastAPI, Redis
 - LLM 연동 : Gemini-2.5-flash, gemini-embedding-001, 자체 sLLM
 - 데이터베이스 : PostgreSQL
- 프론트엔드
 - 프레임워크 : React, Next.js
 - 상호작용 : AJAX 요청을 통한 비동기 통신
 - UI: Radix UI, TailwindCSS

4. LLM 연동 및 데이터베이스 구현

4.1 AI Server 분리

Redis와 데이터베이스를 통한 Back-End - AI 간 통신

1. Back-End는 Redis의 MQ와 hash에 작업 정보와 task id를 등록한 후, 실제 모델에 전달해야할 input data는 데이터베이스의 model_logs에 삽입한다.
 - 1-1. 작업을 전달한 후 Back-End는 Redis의 hash와 주기적으로 통신하며 해당 task id의 작업 진행 상황을 확인한다.

2. AI Worker는 Redis Queue에서 작업을 수령하고, 해당 작업의 task id에 맞는 input data를 데이터베이스에서 가져온다.
3. AI Worker가 작업 정보에 맞게 작업을 AI Engine에 전달하면, 작업을 AI Engine을 통해 수행한다.
4. AI Engine에서의 작업이 완료되면, AI Worker는 완료된 결과를 받아 model_logs의 알맞는 task id의 ai output에 추가하고, hash의 작업 진행 상황을 완료로 변경한다.
5. Back-End에서 조회한 작업 진행 상황이 완료 라면, 해당 task id에 맞는 데이터를 model_logs에서 가져와 남은 작업을 진행한다.

4.2 각 레이어의 처리 권한

LLM과 관련한 처리는 모두 AI Engine에서 수행한다.

AI Server에서는 model_logs를 제외한 DB의 다른 테이블에 대한 입력과 수정은 처리하지 않고 조회 권한만 가진다.

Back-End에서는 Repository Layer를 통해서 모든 DB에 대한 입력과 수정을 수행한다.

Back-End와 AI Server는 직접 통신하지 않고, 모든 통신은 DB와 Redis를 통한다.

model_logs 구조

```
class ModelLog(Base):
    """AI-사용자 상호작용 로그"""
    __tablename__ = "model_logs"
    __table_args__ = {"comment": "sLLM 파인튜닝을 위한 AI-사용자 상호작용 로그"}

    log_seq = Column(Integer, primary_key=True, index=True, comment="로그 고유 식별자 (Sequence)")
    operator_seq = Column(Integer, ForeignKey("users.user_seq"), nullable=True, comment="작업을 수행한 사용자 식별자")
    team_seq = Column(Integer, ForeignKey("teams.team_seq"), nullable=True, comment="작업이 수행된 팀 식별자")
    task_type_code = Column(String(20), nullable=False, comment="작업 단계 1lm_task_type")
    task_id = Column(UUID(as_uuid=True), nullable=True, comment="task_id (uuid)")
    start_task_id = Column(UUID(as_uuid=True), nullable=True, comment="트랜잭션 연결을 위한 시작 task의 id")
    input_data = Column(JSONB, nullable=True, comment="AI 모델에 입력된 프롬프트 또는 데이터 (JSON)")
    ai_output = Column(JSONB, nullable=True, comment="AI 모델이 반환한 결과 데이터 (JSON)")
    user_decision = Column(JSONB, nullable=True, comment="사용자의 최종 수정/승인 데이터 (학습 레이블용)")
    created_at = Column(DateTime, server_default=func.now(), nullable=False)
```

5. 예외 처리 및 보안

5.1 예외 처리

예외 처리 예시

```
ValueError(f"입력 데이터를 찾을 수 없습니다. task_id={task_id}")
ValueError(f"입력 텍스트(text)가 누락되었습니다. task_id={task_id}")
ValueError(f"기존 문서에 대한 유효한 데이터를 찾을 수 없습니다. recipe_seq={recipe_seq}")

if not settings.JWT_SECRET_KEY:
    raise ValueError("JWT_SECRET_KEY is missing in .env or environment")
if not settings.ACCESS_TOKEN_EXPIRE_MINUTES:
    raise ValueError("ACCESS_TOKEN_EXPIRE_MINUTES is missing")
if not settings.AES_SECRET_KEY:
    raise ValueError("AES_SECRET_KEY is missing (required for CryptoService)")
```

5.2 로그 관리

logging 라이브러리를 이용한 로그 관리 및 실행 현황 체크

로그 및 오류 발생 예시

```
2026-02-05 14:41:46 [INFO] [AI Worker] [Worker 실행] 큐 수신 | payload : {'task_id': '82396655-906f-41e9-8899-71ae582d69e2', 'task_type': 'DOC_INDEX', 'task_status': 'PENDING'}
2026-02-05 14:41:46 [INFO] [AI Worker] [Worker 실행] 작업 시작
2026-02-05 14:41:46 [INFO] [AI Engine] [문서 분할] 섹션화 모델 로딩 시작
2026-02-05 14:41:47 [INFO] [sentence_transformers.SentenceTransformer] Use pytorch device_name: mps
2026-02-05 14:41:47 [INFO] [sentence_transformers.SentenceTransformer] Load pretrained SentenceTransformer: app/model_data/paragraph_boundary_minilm
2026-02-05 14:41:49 [INFO] [AI Engine] [문서 분할] 섹션화 모델 로딩 완료
2026-02-05 14:41:50 [INFO] [AI Worker] [문서 조회] 문서 조회 시작 | recipe_seq : 2
2026-02-05 14:41:50 [ERROR] [AI Worker] [Worker 실행] 작업 처리 실패 | task_id : 82396655-906f-41e9-8899-71ae582d69e2 | recipe를 불러올 수 없습니다.
```

5.3 보안 관리

1. 사용자가 문서를 업로드하면, 문서를 AI Server에 전송하기 전에 민감정보를 규칙 기반으로 검색하여 마스킹한다.
2. 해당 정보가 있던 위치는 임의로 생성된 id로 대체가 되고, 탐지된 정보는 env 파일의 team key를 통해 AES-256 방식으로 암호화한다.
3. 또한 SHA-256 방식으로 단방향 암호화 역시 진행하여, 탐지된 정보가 이미 데이터베이스에 저장되어 있는지 확인할 수 있도록 한다.
4. 양방향 - 단방향 암호화를 모두 진행한 정보를 데이터 베이스에 등록한다.
5. 마스킹된 문서를 AI Server에 전달한다.
6. 모든 작업은 마스킹된 형태로 진행되고, 데이터 저장 역시 마스킹 된 형태로 저장된다.
7. 데이터베이스에서 문서를 불러와 사용자에게 출력할 때, 마스킹된 정보가 있다면 해당 마스킹된 정보를 team key를 이용해서 복호화 한 후, 사용자에게 출력한다.

5.4 권한 관리

1. 특정 문서에 대한 권한이 있는 유저만 문서를 검색, 조회, 수정 할 수 있도록 설정

6. 코드 모듈화 및 주석

6.1 모듈화

코드는 각 레이어, 기능 별 별도의 모듈과 클래스로 분리하여 유지보수를 용이하게 하였다.

1. AI Server
 - a. worker.py : Redis와 DB에서 작업을 받아와서, 각 task type에 맞는 작업을 수행하도록 LLMEngine을 호출하고, 작업 결과를 다시 Redis와 DB에 등록하는 작업을 수행
 - b. engine.py
 - i. DocTools : staticmethod로 이루어진 클래스로, 실제 LLM을 호출하여 임베딩 벡터나 요약을 생성하는 등 실제 LLM을 통해서 작업을 수행하는 함수들로 구성
 - ii. LLMEngine : worker에서 호출되는 클래스로, 각 모델을 초기화하고 각 작업에 알맞은 함수를 DocTools에서 가져와 실제 작업을 수행하는 함수들로 구성
2. BackEnd
 - a. Repository Layer : DB에 대한 CRUD를 담당하며, 각 테이블 별로 각각의 모듈과 클래스로 분리되어 쿼리문을 외부 함수에서 입력해주지 않더라도 DB에서 원하는 작업을 수행할 수 있도록 사전에 정의된 함수들을 구현
 - b.